

SyncUs: An Intelligent Job Matching Platform

Group Members:

Name	Student ID	UOW Email
Jenamari Bathan	8565727	jb469@uowmail.edu.au
Gia Han Pham	8519432	ghp726@uowmail.edu.au
Serene Mariam Philip	7803552	smp723@uowmail.edu.au
Tra My Nguyen	8312060	tmn947@uowmail.edu.au
Shakeel Ahmed Mahdhy	8361332	samar565@uowmail.edu.au

Table of Contents

1. Project Overview	3
1.1 Background	3
1.2 Problem Definition	3
1.3 System Objectives	3
1.4 Target Users	4
2. Requirement Analysis	5
2.1 Stakeholder need analysis	5
2.3 Integrated System Workflow & User Stories	6
3. System Analysis	8
3.1 Use Case Diagram Overview	8
3.2 Use Case Diagram: Job Seeker	9
3.3 Use Case Diagram: Employer	12
3.4 Use Case Diagram: Guest User	14
3.5 System Architecture: UML Class Diagram	15
4. Development Model	16
4.1 Methodology Selection & Justification	16
4.2 Scrum Roles & Responsibilities	17
4.3 Workflow Management	18
4.4 Technical Infrastructure & Stack	18
4.5 Definition of Done	19
5. Development Plan	20
5.1 Project Timeline Overview	20
5.2 Sprint-Based Development Plan	20
5.4 Milestone Planning	20
5.5 Gantt Chart and Work Breakdown Structure	21
6. Development Progress	25
6.1 Execution Status of Iterations (Sprint log)	25
6.2 Task Planning and Prioritization (MoSCoW Logic)	25
6.3 Feature Progress and Technical Status	26
6.4 Progress Tracking and Quality Monitoring	28
7. Risk Analysis and Management	29
7.1 Risk Assessment Matrix	29
7.2 Change Management: The Week 8 Pivot Protocol	31

1. Project Overview

1.1 Background

Online recruitment platforms such as Seek and LinkedIn are widely used among job seekers as they play a major role in connecting job applicants and employers. However, these recruitment systems depend mostly on standard filters such as keywords, job titles and years of experience. Although this works well for general recruitment, it often acts as a barrier to recent graduates and professionals in the early stages of their career who possess relevant skills and qualifications but not adequate work experience.

To address this problem, the project develops an Intelligent Talent Matching Platform that aims at improving the job recruitment process. Our system SyncUs is a platform aiming for a Career Ignition in the recruitment market through a deeper personalisation where factors such as an applicant's academic rigor (performance in different units during university) are considered in profile matching beyond basic filters. The system includes an intelligent matching mechanism where both candidates and employers can find suitable matches with less effort.

1.2 Problem Definition

Many traditional job recruitment platforms mainly rely on filtering based on basic standards such as minimum number of work experience, qualification, job title etc. Such filtering becomes a disadvantage for those new to the workforce such as a student seeking a suitable internship or a recent graduate with necessary qualifications trying to land in a suitable role. This can result in applicants missing valuable opportunities and employers failing to hire strong candidates who can be potential assets in the workforce.

An automated matching system that connects candidate profiles with job requirements following a Competency-Based filtering rather than Tenure-Based filtering can iron out the existing issue thereby making the recruitment process more effective. Through this approach, a capable individual with strong academic performance and project experience won't get excluded for simply not meeting the experience requirement. The system analyses user data and provides relevant recommendations for applicants and employers.

1.3 System Objectives

The main objectives of the system are:

- To develop a web-based platform that connects students and candidates new to the workforce with employers efficiently.
- To allow candidates to create profiles and enter candidate information, upload resume and browse job opportunities using keyword function.
- To allow applicants to highlight their academic achievements, project experiences and technical skills (e.g., GitHub repositories).
- To allow employers to create job postings, detailed job descriptions and search for candidates.
- To allow employers to filter candidates using search functions based on job description.
- To allow employers to specify required skills and technologies (e.g., React, Supabase)

- To implement a competency-based matching system that recommends suitable jobs and candidates based on skills rather than experience alone.
- To develop a two-way matching mechanism that recommends
 - Top-K (K=10) relevant jobs for candidates
 - Top-N (N =10) best candidates for employers
- To improve the accuracy of job searching and talent acquisition through structured data and automated matching.

1.4 Target Users

The target users are described through personas to better understand the characteristics, challenges and requirements needed for the system.

Alex (Student)

Demographics	Alex is a 22-year-old final-year student studying Computer Science at the University of Wollongong. He lives in NSW and is actively applying for internships and full-time positions.
Psychographics	Alex is eager to start a career in the tech industry and is looking for opportunities to showcase his skills that could help him learn and grow professionally.
Behavioural Data	Alex frequently uses platforms such as Seek and LinkedIn to search for suitable job opportunities. To demonstrate his technical skills, he also maintains a GitHub portfolio and works on personal projects.
Pain Point	The main challenge faced by Alex is the lack of formal work experience that prevents him from finding a suitable role. Despite having strong academic skills and project experience, he is often not considered as many positions require “years of experience”.
Requirements	He requires a platform where he can highlight his technical skills, academic performance (UOW units) and projects that allows employers to assess his capabilities beyond just experience.
Media Used	Alex mostly relies on platforms such as Seek, LinkedIn, university portals and job boards to look for opportunities.

Sarah (Employer)

Demographics	Sarah is 30 years old and owns a startup company based in NSW. She has a small development team working in the company and is responsible for hiring new talent to join the company.
Psychographics	Sarah is looking for candidates with good practical skills and efficiency. She prefers a candidate who can join immediately and contribute to the company's technical needs.
Behavioural Data	She posts job listings mainly on platforms such as LinkedIn and Seek and goes through each application manually. She often uses the filter feature to sort candidates based on required skills and technologies.

Pain Point	Sarah receives a large number of applications and finds it challenging and time-consuming to review every application manually. She has difficulties in identifying the right candidate with suitable skills especially when most candidates lack direct work experience but are still capable.
Requirements	She is looking for a platform that gives recommendations for candidates based on skills (e.g., React), academic performance, and project experience rather than the number of years of work experience.
Media Used	Uses platforms such as LinkedIn, Gmail and other recruitment platforms to communicate with suitable candidates.

2. Requirement Analysis

2.1 Stakeholder need analysis

The identified stakeholder needs are identified and translated to key functional requirements to ensure all requirements in the system are covered.

2.1.1 Candidate-Centric Requirements

Stakeholder Needs	Functional Requirements
Competency-Based Profiling	Structured Input: Full name, Contact Information, Education Major/Field of study, Years of Experience, Academic units, and project link
Resume Management	Ability to upload, store, and manage up to three versions of a resume (PDF/DOCX) and designate a primary version for applications
Discovery without Data Walls	Ability for unauthenticated or incomplete profiles to view a "General Feed" of all job postings
Automated Matching (K=10)	An intelligent matching engine that ranks the 10 most relevant jobs based on his major and skills.
Targeted Discovery	Search and filter jobs based on Job Description to ensure results are based on role requirements rather than just job titles.
Automated Alerts	System-generated notifications when a new job matching the candidate's profile is posted or when an employer interacts with their application.
Application tracking and notifications	A personalized dashboard where candidates can monitor the real-time status of their applications (Pending, Viewed, Shortlisted, Matched or Rejected).

2.1.2 Employer-Centric Requirements

Stakeholder Needs	Functional Requirements
Keyword-Rich JD Creation	A posting tool that mandates the inclusion of Job Title, Company Info, Required Skills, Education Level, Years of Experience, Work Mode (Remote/On-site/Hybrid), and Location.
Automated Candidate Screening	Top-N (N=10) Matching: generate a list of the Top-10 most suitable candidates based on the

	specific requirements of a job posting.
Advanced Candidate Sourcing	A multi-criteria filtering system allowing employers to browse all profiles by specific technologies, degrees, or academic background.
Communication channel	The JD creation form must include a mandatory, validated Email Address field that is rendered as a clickable mailto: link on the public posting.
Application management	A dashboard to review applicant profiles, download resumes, and update application statuses (Applied, Shortlisted, Matched or Rejected)

2.3 Integrated System Workflow & User Stories

This section integrates system workflows with User Stories (US), offering an end-to-end view of functionality from each stakeholder's perspective. The stories follow the INVEST framework, ensuring that each feature delivers a distinct benefit and is technically verifiable through Acceptance Criteria (AC).

US1 – Profile Creation As a job seeker, I want to input my academic and professional details so the system can match me with compatible roles.

- AC: Mandatory fields (name, contact, education, major) validated before submission; data stored in structured format; profile editable at any time.

US2 – Resume Management As a job seeker, I want to upload and manage multiple resumes so I can tailor applications.

- AC: Max 3 resumes per user; PDF/DOCX only; max 5MB; one resume designated as primary; files securely stored and retrievable.

US3 – General Job Feed As a job seeker, I want to browse all job postings before completing my profile so I can explore the market freely.

- AC: General feed accessible to incomplete/unauthenticated profiles; displays title, company, location, and type; remains available if recommendation engine is offline.

US4 – Keyword Job Search As a job seeker, I want to search for roles using keywords in the job description.

- AC: Search matches the Job Description field only (not job title); supports partial matches (e.g., "React" returns "React.js"); displays "No results found" when no match exists.

US5 – Smart Job Recommendations As a job seeker, I want to receive Top-10 job recommendations based on my profile.

- AC: Exactly 10 recommendations generated from profile data (skills, education, projects); rankings displayed; recommendations refresh when profile is updated.

US6 – Apply & Track Applications As a job seeker, I want to apply for jobs and monitor my application status.

- AC: Resume selection required at application; statuses (Applied / Shortlisted / Denied) visible in dashboard.

US7 – Withdraw Application As a job seeker, I want to withdraw an application before a final decision is made.

- AC: Withdrawal available while decision is pending; status updated to "Withdrawn."

US8 – Notifications As a job seeker, I want to receive alerts when employers interact with my profile or new matching jobs are posted.

- AC: Notifications triggered on profile view, status change, and new relevant postings; displayed in notification panel.

US9 – Company Insights As a job seeker, I want to view company ratings and response times to make informed decisions.

- AC: Ratings and response-time indicators (e.g., "Responds within 3 days") visible on job listings.

US10 – Skill Trends As a job seeker, I want to access trending skill suggestions to improve my employability.

- AC: Trending skills displayed in feed based on profile; updated from current job market data.

US11 – Create Job Posting As an employer, I want to create detailed job postings so that suitable candidates can apply.

- AC: Mandatory fields: title, company info, description, skills, location; optional: experience, work mode; postings editable after creation; only published postings visible to candidates.

US12 – Branded Company Page As an employer, I want a career page to showcase my organisation.

- AC: Employers can add description, logo, and website link; page visible to candidates.

US13 – View Candidate Profiles As an employer, I want to review applicant profiles to assess suitability.

- AC: Profiles display skills, education, and project experience; only completed profiles surfaced.

US14 – Search & Filter Candidates As an employer, I want to search and filter candidates by skills, education, and experience.

- AC: Keyword search matches skills and projects; multiple simultaneous filters supported; results returned promptly.

US15 – Shortlist Candidates As an employer, I want to shortlist candidates for later review.

- AC: Employers can mark candidates "Shortlisted"; shortlisted candidates saved in a separate list.

US16 – Smart Candidate Recommendations As an employer, I want Top-10 candidate recommendations based on my job description.

- AC: Exactly 10 candidates recommended by skills, academic performance, and JD requirements; list updates when JD is revised.

US17 – Manage Application Statuses As an employer, I want to update application statuses to manage the hiring pipeline.

- AC: Status options: Applied / Shortlisted / Denied; candidates notified immediately on status change.

US18 – Compliance Record-Keeping As an employer, I want application records maintained securely for compliance purposes.

- AC: Applications stored securely; full history accessible to employers; data accessible for audit and compliance review.

3. System Analysis

3.1 Use Case Diagram Overview

The Job Portal System was modelled using a **use case-driven approach** to capture functional requirements from multiple user perspectives. The system is divided into four primary actors:

- **Job Seeker** – interacts with the system to create profiles, search for jobs, receive recommendations, and manage applications.
- **Employer/Recruiter** – manages job postings, reviews applicants, and processes hiring decisions.
- **Guest User** – limited access to view public job listings and platform information.

The system architecture is designed using **UML Use Case Diagrams**, with:

- **<<include>>** used for **mandatory sub-processes** (e.g., validation, selection, authentication).
- **<<extend>>** used for **optional or conditional behaviours** (e.g., filtering, withdrawal, recommendations refinement).
- Internal system actors such as the **Recommendation Engine** modelled as a **supporting system actor**, representing automated decision-making processes.

This modelling approach ensures:

- Clear separation between **user goals and system logic**
- Reusable system functions through included behaviours
- Extensible functionality for intelligent features such as job recommendations

3.2 Use Case Diagram: Job Seeker

The use case diagram was refined using <<include>> and <<extend>> relationships to distinguish between mandatory system processes (e.g., validation, submission) and optional user-driven actions (e.g., filtering, withdrawing applications). Additional system-level use cases were introduced to represent internal processing logic such as recommendation matching and data validation.

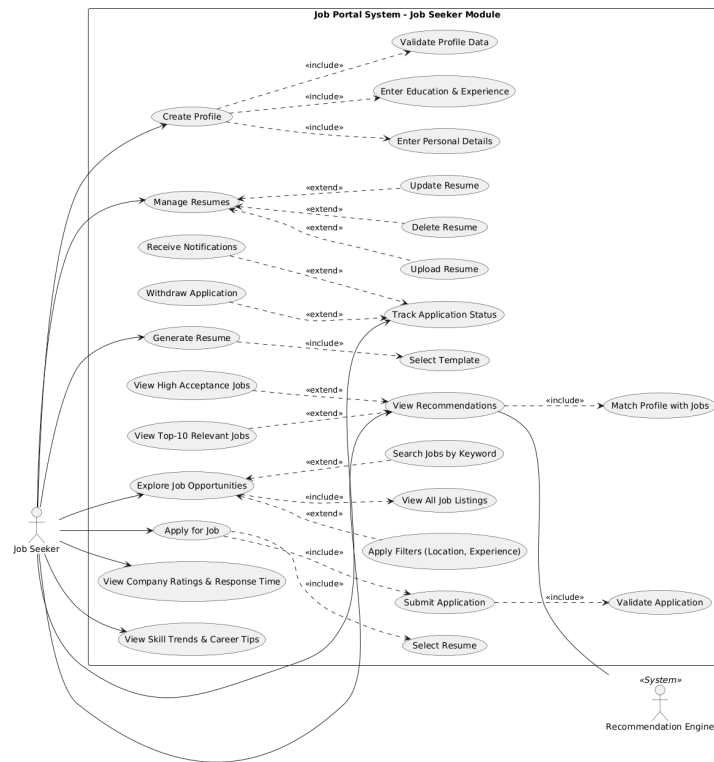


Figure 1: Job Seeker Use Case Diagram

Field	Description
Use Case Name	Create Profile
Description	Allows the user to build a digital presence by entering personal and professional data.
Pre-condition	The user is registered and logged into the system.
Post-condition	Profile is saved and visible to potential employers.
Main Success Scenario	1. The user selects "Create Profile." 2. System prompts for personal details (e.g. name, contact). 3. The user enters professional details (e.g. experience, education, skills). 4. The user saves the profile. 5. The system validates data and confirms successful creation.

Extensions	3a. The user chooses to "Generate Resume" using a template (via <<include>>).
-------------------	---

3.2.2 Profile Creation: Manage Resumes

Field	Description
Use Case Name	Manage Resumes
Description	Enables the user to upload, store, and organize multiple versions of their resume.
Pre-condition	The user has an active account and is on the "My Documents" page.
Post-condition	Resumes are stored in the cloud and available for job applications.
Main Success Scenario	1. The user selects "Manage Resumes." 2. The system displays a list of currently uploaded resumes. 3. The user chooses to upload a new file or edit metadata of an existing one. 4. The user confirms the action. 5. System updates the repository.
Extensions	3a. User selects "Delete Resume" to remove an outdated version (via <<extend>>).

3.2.3 Job Discovery: Search & Filter Jobs

Field	Description
Use Case Name	Search & Filter Jobs
Description	Allows users to find specific job openings based on keywords or specific constraints.
Pre-condition	The user is on the Job Search dashboard.
Post-condition	A refined list of job postings is displayed to the user.
Main Success Scenario	1. The user enters a keyword (e.g, "Software Engineer") into the search bar. 2. The system retrieves all matching active job postings. 3. The system displays the results sorted by relevance.
Extensions	2a. The user applies filters such as "Location," "Salary Range," or "Experience Level" (via <<extend>>). 2b. The system refreshes results to meet filter criteria.

3.2.4 Smart Recommendations: View Recommendations

Field	Description
Use Case Name	View Recommendations
Description	Provides a curated list of jobs tailored to the user's specific skill set and career history.
Pre-condition	The user has completed at least 80% of their profile details.
Post-condition	The user views a personalized "Recommended for You" feed.
Main Success Scenario	1. The user navigates to the "Recommendations" tab. 2. The system triggers the Recommendation Engine to analyze the user's profile. 3. The system displays the Top-10 (\$K=10\$) most relevant job matches.
Extensions	3a. User toggles "High Chance of Acceptance" to see roles where their profile matches >90% of requirements (via <<extend>>).

3.2.5 Application Management: Apply for Job

Field	Description
Use Case Name	Apply for Job
Description	The process of submitting a formal interest in a specific job posting.
Pre-condition	The user has found a job posting and has at least one resume uploaded.
Post-condition	Application is sent to the employer; status is updated to "Pending."
Main Success Scenario	1. The user clicks "Apply Now" on a job posting. 2. The system prompts the user to select a resume (via <<include>>). 3. The user attaches a cover letter (optional). 4. The user submits the application. 5. The system notifies the Employer and logs the application in the user's "Track Progress" dashboard.
Extensions	5a. System triggers a notification to the user confirming the submission (via <<include>>).

3.2.6 Application Management: Track Progress

Field	Description
-------	-------------

Use Case Name	Track Progress
Description	Allows the user to monitor the status of all submitted applications.
Pre-condition	The user has submitted at least one application.
Post-condition	The user is informed of the current stage of their candidacy.
Main Success Scenario	1. The user visits the "My Applications" dashboard. 2. The system displays a list of applications with statuses (e.g., Applied, Viewed, Interviewing). 3. The user clicks on a specific application to see detailed history.
Extensions	3a. The user decides they are no longer interested and selects "Withdraw Application" (via <<extend>>).

3.3 Use Case Diagram: Employer

The Employer actor focuses on talent acquisition, posting management, and candidate evaluation.

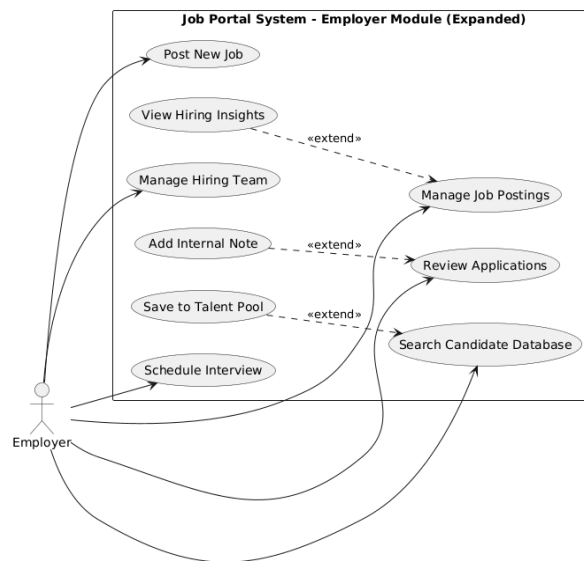


Figure 2: Job Seeker Use Case Diagram

3.3.1 Job Management: Post New Job

Field	Description
Use Case Name	Post New Job
Description	Allows employers to list new vacancies with specific requirements.
Pre-condition	The employer account is verified and active.

Post-condition	Job posting is live and searchable by Job Seekers.
Main Success Scenario	1. The employer selects "Post Job." 2. The system provides a form for title, description, and requirements. 3. The employer sets screening questions. 4. The system validates the listing for compliance. 5. Job is published.
Extensions	3a. System suggests "Auto-fill" based on previous templates (via <<extend>>).

3.3.2 Candidate Selection: Review Applications

Field	Description
Use Case Name	Review Applications
Description	Enables recruiters to screen applicants and move them through the hiring funnel.
Pre-condition	At least one candidate has applied for a live posting.
Post-condition	Candidate status is updated (e.g., Shortlisted, Rejected).
Main Success Scenario	1. Employer views "Application Inbox." 2. The system displays candidate profiles and match scores. 3. Employer views resume/cover letter. 4. Employer updates status to "Shortlisted." 5. The system triggers a notification to the Job Seeker.
Extensions	2a. System highlights "Top Matches" using the Ranking Engine (via <<include>>).

3.3.3 Talent Sourcing: Search Candidate Database

Field	Description
Use Case Name	Search Candidate Database
Description	Allows recruiters to proactively find talent by searching through public profiles rather than waiting for applications.
Pre-condition	The employer has an active subscription/access level for database searching.
Post-condition	A list of potential candidates is displayed for outreach.
Main Success Scenario	1. The employer enters criteria (e.g., "React Developer", "New York"). 2. The system filters all public profiles matching the criteria. 3. The employer reviews "Top Matches" based on skill relevance.
Extensions	3a. Employer selects "Save to Talent Pool" for future vacancies (via <<extend>>).

3.3.4 Recruitment Analytics: View Hiring Insights

Field	Description
Use Case Name	View Hiring Insights
Description	Provides data on job posting performance, such as click-to-apply rates and applicant demographics.
Pre-condition	The employer has at least one active or past job posting.
Post-condition	Recruiter understands the effectiveness of their job descriptions and sourcing channels.
Main Success Scenario	1. Employer selects "Analytics Dashboard." 2. The system generates a report on view counts, application volume, and time-to-hire. 3. The employer exports the data for internal stakeholders.
Extensions	3a. Employer selects "Save to Talent Pool" for future vacancies (via <<extend>>).

3.3.5 Team Collaboration: Manage Hiring Team

Field	Description
Use Case Name	Manage Hiring Team
Description	Allows a primary recruiter to invite team members (interviewers) to review specific candidates.
Pre-condition	The employer is the "Admin" or "Owner" of the company profile.
Post-condition	Multiple users can leave feedback on a single candidate.
Main Success Scenario	1. The employer selects a specific job posting. 2. The employer clicks "Add Collaborator" and enters an email address. 3. The system sends an invite and grants restricted access to that job's applicants.
Extensions	3a. Employer selects "Save to Talent Pool" for future vacancies (via <<extend>>).

3.4 Use Case Diagram: Guest User

The Guest User is an unauthenticated actor exploring the platform's value.

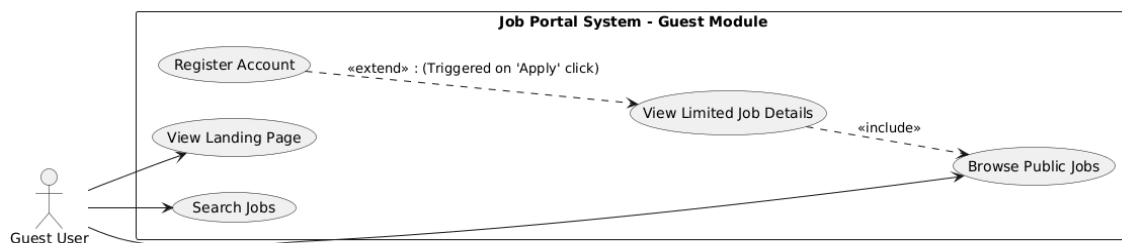


Figure 3: Guest User Use Case Diagram

3.4.1 Entry Logic: Landing Page Exploration

Field	Description
Use Case Name	Browse Public Postings
Description	Allows non-registered users to view the scope of available jobs.
Pre-condition	None (Public access).
Post-condition	Users are prompted to register to apply.
Main Success Scenario	1. The guest user visits the homepage. 2. The guest user performs a "Quick Search" by keyword. 3. The system displays limited job details.
Extensions	3a. Guest clicks "Apply" → System redirects to "Register/Login" (via <<include>>).

3.5 System Architecture: UML Class Diagram

The following diagram illustrates the relationship between the core services and the data entities they manage. This structure supports the "Modular Monolith" approach by defining clear boundaries for each service.

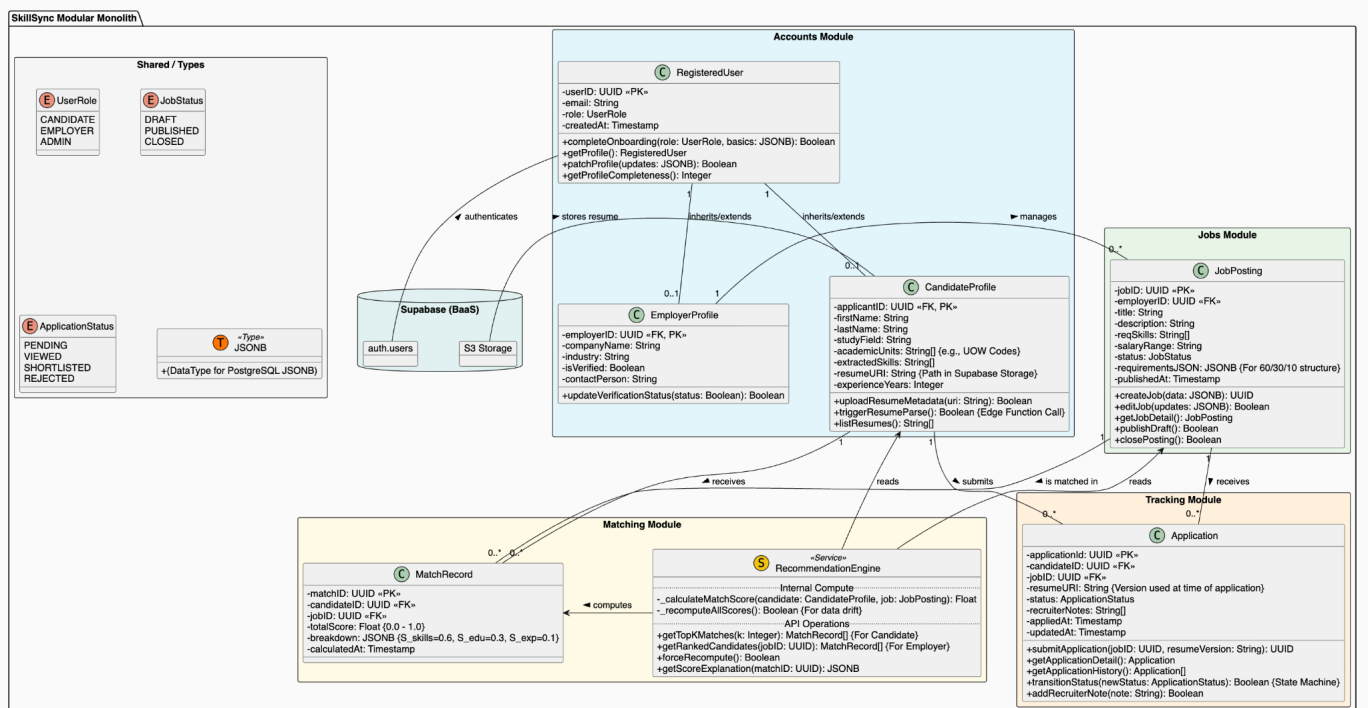


Figure 4: System UML Class Diagram

4. Development Model

4.1 Methodology Selection & Justification

A formal constraint audit was conducted to identify the most suitable methodology for the 10-week delivery window (Week 4–13):

Constraint	Audit observation	Methodological Requirement
Schedule	10-week delivery window (Week 4 to Week 13) with a guaranteed change in Week 8.	Must be Iterative and Flexible .
Team	A multidiscipline team of 5 students with varying schedules and remote collaboration.	Must have Clear Roles and Low Overhead .
Stakeholders	Opportunity to seek feedback during weekly consultation hours and biweekly lab hours	Support Frequent Feedback cycles

The table below summarises the evaluation of candidate methodologies:

Methodology	Alignment	Reason
Waterfall	Poor	Linear structure incompatible with the guaranteed Week 8 change
Spiral	Poor	Formal risk-analysis cycles are excessive for a 10-week scope
Extreme Programming	Poor	Pair programming impractical given remote collaboration and varying schedules
DevOps	Poor	CI/CD infrastructure setup would exceed total project duration
Lean	Poor	"Eliminate documentation" principle conflicts with formal reporting requirements
RAD	Partial	Prototyping is useful, but constant user feedback is infeasible with bi-weekly tutor access only
FDD	Partial	Feature focus is useful but too granular for a small team managing architecture and UI/UX simultaneously
Kanban	Partial	Good for task visualisation, but lacks Sprint time-boxing, risking scope creep

Selected Methodology: Scrum. Scrum best satisfies all three constraints through 1-week Sprints that create natural pivot points, a structured backlog with MoSCoW prioritisation, and defined roles that suit a small cross-functional team (Drumond 2024). Key justifications:

- **Internal Discipline:** Sprint Reviews act as internal deadlines, maintaining velocity without external pressure.

- **Task Management:** Taiga manages the backlog and burn-down charts, enabling velocity monitoring and de-scoping of non-essential features if the team falls behind.
- **Iterative Resilience:** 1-week Sprints create 10 pivot points, allowing the Week 8 requirement change to be treated as a scheduled Backlog Refinement event rather than a disruption. The team can re-estimate the backlog within 48 hours of the announcement.
- **Risk Front-Loading:** High-risk components (e.g., the 60/30/10 Matching Algorithm) are prioritised in Sprints 2–4, ensuring the core system is stable before the Week 8 change arrives.

4.2 Scrum Roles & Responsibilities

The following section defines the formal governance structure and individual accountabilities established for the development of the **SyncUs**-Intelligent Talent Matching Platform. To ensure project success within a compressed ten-week lifecycle, roles have been allocated based on technical specialization, leadership competency, and alignment with the specific architectural requirements of the React/Supabase ecosystem.

Role	Assignee	Justification	Key Responsibilities
Scrum Master	Gia Han Pham	Selected for proficiency in organizational governance and high-level task management; Selected for their ability to synthesize complex requirements into actionable sprint tasks and their proficiency in Agile project management tools	Maintain Taiga board; facilitate sprint planning, reviews, and retrospectives; monitor burn-down velocity; identify and eliminate blockers
Product Owner	Jenamari Bathan	Expertise-centric design; possesses the analytical depth to prioritize the Product Backlog .	Defining User Stories and Acceptance Criteria (AC) Validating that increments satisfy the project brief.
Development Team	Serene Mariam Philip Tra My Nguyen Shakeel Ahmed Mahdhy	Self-organized, cross-functional development team that can work autonomously to manage and complete the tasks	Matching engine, UI/UX development, database schema management; ensure all code meets the Definition of Done

To eliminate bottlenecks during the critical Week 8–9 pivot, the team operates under the following governance protocol (Kulkarni, 2023):

Governance Strategy	Tactical Implementation	Targeted Bottleneck
Information Collection	Enforcing a "Ready-to-Dev" Gate where no ticket enters the sprint without explicit Acceptance Criteria (AC) .	Requirement Vagueness: Prevents developers from stalling due to unclear functional logic or shifting priorities.
Intensified Collaboration	Transitioning from weekly syncs to Daily Stand-ups during the Week 8-9 pivot window.	Communication Silos: Prevents "Work Overlap" or "Obsolete Development" caused by developers working on outdated requirements.
Objective Alignment	Anchoring every 1-week sprint with a measurable Sprint Goal and a shared vision for the Alex and	Strategic Drift: Prevents the team from losing focus on core customer value when market dynamics

	Sarah personas.	(requirements) shift.
Impediment Shielding	Implementing a 1-hour solo troubleshooting limit before formally flagging technical blockers to the Scrum Master.	Rabbit-Hole Delays: Prevents significant time loss on environmental or configuration issues
Quality Governance	Mandatory Peer-Review Loop via GitHub Pull Requests before any ticket is marked "Done" on Taiga.	Integration Debt: Prevents code conflicts and system instability during the final Weeks 12–13.

4.3 Workflow Management

To maintain high velocity and code integrity within a compressed 10-week lifecycle, the team operates under a structured **"Branch-Commit-Verify"** framework. Central to this is the implementation of a **Three-Tier Branching Strategy**, which provides a "Safety Nest" for the evolution of the SyncUs codebase. Unlike a single-branch workflow, our three-tier system isolates development risks through a dedicated staging environment:

Methodology	Reason
Main Branch (Production)	Source of truth; always deployable for tutor consultations and demos
Dev Branch (The Safety Nest)	Primary integration gateway; synchronises frontend and backend before promoting to main, preventing integration debt
Feature Branche	All development work; no direct commits to main or dev; enables individual accountability and clean rollbacks

Execution discipline is enforced through three practices: a **strict local sync** (pull-merge-push before coding to resolve conflicts locally), **atomic commits** (frequent granular commits allowing rollback to a stable HEAD), and **mandatory Pull Requests** where the Definition of Done is peer-verified before any merge to dev.

4.4 Technical Infrastructure & Stack

The **SyncUs** platform utilizes a modern, cloud-native stack chosen for its ability to handle complex relational data (PostgreSQL) and rapid frontend iteration (React). By migrating to a Serverless BaaS (Backend-as-a-Service) model via Supabase, the architecture minimizes infrastructure management while maximizing the performance of the matching engine.

The Core Technology Stack

Layer	Technology	Justification
Frontend	React.js	Powers a dynamic, state-driven UI necessary for the Reciprocal Handshake interface. It allows for real-time updates on the Candidate Dashboard and seamless transitions between structured form inputs and recommendation views.

Backend & Logic	Python FastAPI	Serves as the central "Matching Engine." Python's computational power is used to execute the 60/30/10 weighted matching algorithm, providing high-performance candidate scoring, "Top-K" and "Top-N" shortlisting.
Database	PostgreSQL (via Supabase)	Provides the robust relational integrity required to manage complex Many-to-Many relationships between Job Seekers and Employers. Supports advanced filtering for "Top-K" and "Top-N" candidate shortlisting.
Authentication & Storage	Supabase Auth & Storage	Handles secure user authentication and S3-compatible resume storage. Implements Row-Level Security (RLS) to ensure that student profiles and project links are only accessible to authorized recruiters.
Design Tool	Figma	Used for high-fidelity wireframing and prototyping to ensure UX consistency before development. It also ensures that the "Screening Fatigue" solution is user-tested.
Version Control & Workflow	GitHub	Facilitates the Scrum methodology through collaborative branching and pull requests. Manages the version history of SQL schemas, Edge Functions, and React components during 1-week sprints.

4.5 Definition of Done

A feature is only marked "Done" when all of the following are satisfied:

Development & Code Quality

- Code written, refactored, and compliant with team coding standards
- Peer-reviewed via Pull Request; all comments resolved before merge to dev
- No critical linting errors

Testing & Verification

- Unit tests written for all logic functions with $\geq 80\%$ coverage, including edge cases
- Integration testing confirms correct interaction with Supabase/PostgreSQL, including RLS policy enforcement and CRUD data integrity
- UAT manual walkthrough confirms all Acceptance Criteria are met and UI is responsive and consistent with approved Figma prototypes

Documentation & Approval

- Architectural changes, schema updates, and API modifications documented in the repository README
- User-facing help guides or onboarding flows updated if the feature alters the user journey
- Product Owner sign-off obtained confirming business requirements are satisfied
- Code merged to main and verified functional in the staging environment

5. Development Plan

5.1 Project Timeline Overview

The SyncUs platform will be developed over a 10-week period (Week 4 – Week 13) using Scrum methodology with 1-week sprints. Each sprint delivers a functional increment aligned with user stories and acceptance criteria defined in the requirement analysis.

Key approach:

- Iterative development with weekly deliverables
- Early development of core matching engine (high-risk component)
- Continuous feedback from tutors during consultation sessions
- Flexibility to adapt to Week 8 requirement change

5.2 Sprint-Based Development Plan

The sprint-based iterative structure allows the team to track progress effectively and maintain consistent productivity.

Phase	Sprint	Week	Sprint Goal	Key Deliverables
Planning & Requirement Finalization	Sprint 1	Week 4	Requirements & Planning	User stories, backlog, use case diagrams
System Design & Architecture	Sprint 2	Week 5	System Design	UI wireframes, DB schema, architecture setup
Core Development	Sprint 3	Week 6	Core Backend Setup	API structure, authentication system
	Sprint 4	Week 7	Matching Engine (Core Logic)	Top-K job recommendation system
	Sprint 5	Week 8	Requirement Adaptation	Adjust features based on changes
Feature Expansion	Sprint 6	Week 9	Job & Profile Features	Job posting, profile creation
	Sprint 7	Week 10	Search & Filtering	Job search, candidate filtering
Integration & Testing	Sprint 8	Week 11	Integration	Connect frontend + backend
	Sprint 9	Week 12	Testing & Optimization	Bug fixes, performance tuning
Final Deployment & Presentation	Sprint 10	Week 13	Final Delivery	Deployment + presentation

5.4 Milestone Planning

Milestones act as key checkpoints throughout the project, helping the team evaluate progress and ensure alignment with project goals. Each milestone represents the completion of a significant phase in development.

The following milestones guide the overall project execution:

Milestone	Week	Description
M1: Requirements Completion	Week 4	All user stories and use cases finalized
M2: System Design Approval	Week 5	Architecture and UI confirmed
M3: Core Engine Implementation	Week 7	Matching algorithm functional
M4: Feature Completion	Week 10	All major features implemented
M5: System Integration	Week 11	Full system connected
M6: Testing Completion	Week 12	System validated with UAT
M7: Final Submission	Week 13	Project demo + report

5.5 Gantt Chart and Work Breakdown Structure

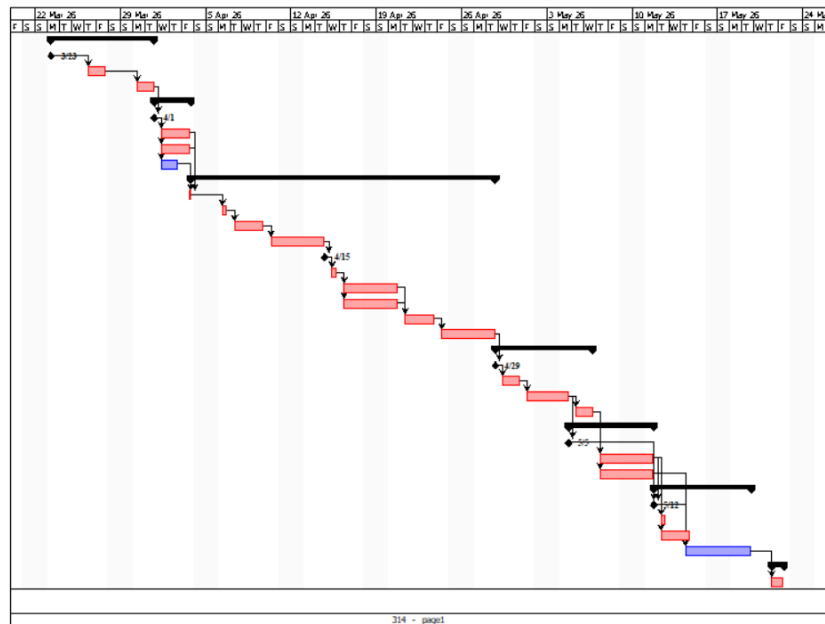


Figure 5: Project Gantt Chart

1.1 Project Setup

Description	Set up project environment, tools, and repositories
Deliverables	GitHub repository, Taiga board, project structure

1.2 Project Start (Milestone)

Description	Marks official project commencement
Deliverables	Project timeline activation

Responsible	Team
Dependencies	None

Responsible	Team
Dependencies	1.1

2.1 Requirements Gathering

Description	Collect and analyze system requirements
Deliverables	Requirement notes, feature list
Responsible	Team
Dependencies	1.2

2.2 User Stories & Use Cases

Description	Define system functionality using user stories
Deliverables	Use case diagrams, user stories
Responsible	Jena
Dependencies	2.1

2.3 M1: Requirements Complete

Description	Approval of all requirements
Deliverables	Final requirement document
Responsible	Team
Dependencies	2.2

3.1 System Architecture Design

Description	Design system structure and modules
Deliverables	Architecture diagram
Responsible	Shakeel
Dependencies	2.3

3.2 UI/UX Wireframing (Figma)

Description	Create UI layouts and user flow
Deliverables	Wireframes
Responsible	My, Serene
Dependencies	2.3

3.3 Database Schema Design

Description	Design database structure and relationships
Deliverables	ER diagram, schema
Responsible	Jena, Han
Dependencies	2.3

3.4 M2: Design Approved

Description	Approval of system design
Deliverables	Approved design
Responsible	Team
Dependencies	3.1, 3.2, 3.3

4.1 Backend Setup (FastAPI)

Description	Initialize backend framework
Deliverables	FastAPI project setup
Responsible	Team
Dependencies	3.4

4.2 Account module

4.3 Matching Engine Development

Description	Implement login, authentication and profile management
Deliverables	Auth and Profile APIs
Responsible	Serene, My
Dependencies	4.1

Description	Develop job recommendation logic
Deliverables	Matching algorithm, API
Responsible	Jena
Dependencies	4.1

4.4 M3: Matching Engine Ready

Description	Completion of matching engine
Deliverables	Functional matching system
Responsible	Jena
Dependencies	4.3

4.5 Requirement Adjustment

Description	Adjust system based on new requirements
Deliverables	Updated features
Responsible	Team
Dependencies	4.4

4.6.1 Job Module

Description	Enable job creation, searching and filtering functionality
Deliverables	Job APIs
Responsible	Shakeel, Han
Dependencies	4.1

4.6.2 Tracking Module

Description	Enable job creation, searching and filtering functionality
Deliverables	Job APIs
Responsible	Shakeel, Han
Dependencies	4.1

4.6.4 Recommendation Dashboard

Description	Display job recommendations
Deliverables	Dashboard UI
Responsible	Team
Dependencies	4.6.1

5.1 M4: Features Complete

Description	Completion of all features
Deliverables	Functional system
Responsible	Team
Dependencies	4.6.4

5.2 Frontend-Backend Integration

5.3 System Testing

Description	Connect frontend with backend
Deliverables	Integrated system
Responsible	Team
Dependencies	5.1

Description	Test integrated system
Deliverables	Test reports
Responsible	Team
Dependencies	5.2

5.4 Bug Fixing & Optimization

Description	Fix bugs and improve performance
Deliverables	Stable system
Responsible	Team
Dependencies	5.3

6.1 M5: System Integrated

Description	Integration phase completion
Deliverables	Integrated system
Responsible	Team
Dependencies	5.4

6.2 User Acceptance Testing

Description	Validate system with users
Deliverables	UAT feedback
Responsible	Team
Dependencies	6.1

6.3 Final Improvements

Description	Apply final refinements
Deliverables	Improved system
Responsible	Team
Dependencies	6.2

6.4 M6: Testing Complete

Description	Completion of testing phase
Deliverables	Tested system
Responsible	Team
Dependencies	6.3

7.1 Final Deployment

Description	Deploy system
Deliverables	Live/demo application
Responsible	Team
Dependencies	6.4

7.2 Documentation Completion

Description	Finalize documentation
Deliverables	Final report
Responsible	Team
Dependencies	6.4

7.3 Final Presentation Preparation

Description	Prepare presentation
Deliverables	Slides, demo
Responsible	Team
Dependencies	7.1, 7.2

7.4 M7: Final Submission

8.1 Project End

Description	Submit final project
Deliverables	Submission package
Responsible	Team
Dependencies	7.3

Description	Marks project completion
Deliverables	Completed system
Responsible	Team
Dependencies	7.4

6. Development Progress

6.1 Execution Status of Iterations (Sprint log)

As of the current reporting date (April 16, Week 7), the team has successfully transitioned from planning into core system development with milestone **M1: Requirements Completion** and **M2: System Design Approval** achieved.

- **Sprint 1 & 2 (Weeks 4–5) – Planning & Design: COMPLETED.** Finalized 19 INVEST-compliant user stories and documented all primary use cases (M1) . Approved high-fidelity Figma wireframes and finalized the technical architecture (M2).
- **Sprint 3 (Week 6) – Core Backend Setup: COMPLETED.** Successfully initialized the **Python FastAPI** framework and established a secure **connection to the Supabase** cloud environment .
- **Sprint 4 (Week 7) – Database & Structure: IN-PROGRESS.** The team is currently organizing the **modular monolithic** project structure on GitHub and finalizing the relational database schema via the **Supabase Dashboard**

6.2 Task Planning and Prioritization (MoSCoW Logic)

The MoSCoW method is a simple way to prioritise project requirements by dividing them into four categories based on importance. It helps the team focus on what is essential for the MVP while managing time and resources effectively.

Priority	Criteria
MUST-HAVE (Essential)	• Features the system cannot function without. • If these are missing, the project fails.
SHOULD-HAVE (Important)	• Features the system cannot function without. • If these are missing, the project fails.
COULD-HAVE (Optional)	• Valuable features that improve the system but are not critical for the MVP. • Can be added if time allows.
WON'T-HAVE (Out of Scope)	• Features intentionally excluded from this release due to time or complexity. • May be considered in future versions.



Figure 6: MoSCoW Logic

6.3 Feature Progress and Technical Status

As reflected in the project repository (**GitHub: <https://github.com/shakeelmahdhy/SyncUs.git>**), the architecture is divided into two primary layers: **FastAPI/Supabase (Backend)** and **React (Frontend)**.

shakeelmahdhy / SyncUs

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Files

main

Go to file

backend/app

api

core

db

modules

accounts

jobs

matching

tracking

utils

main.py

SyncUs / backend / app

shakeelmahdhy initial commit 29894cc · 1 hour ago History

Name	Last commit message	Last commit date
..		
api	initial commit	1 hour ago
core	initial commit	1 hour ago
db	initial commit	1 hour ago
modules	initial commit	1 hour ago
utils	initial commit	1 hour ago
main.py	initial commit	1 hour ago

Figure 7: GitHub Repository

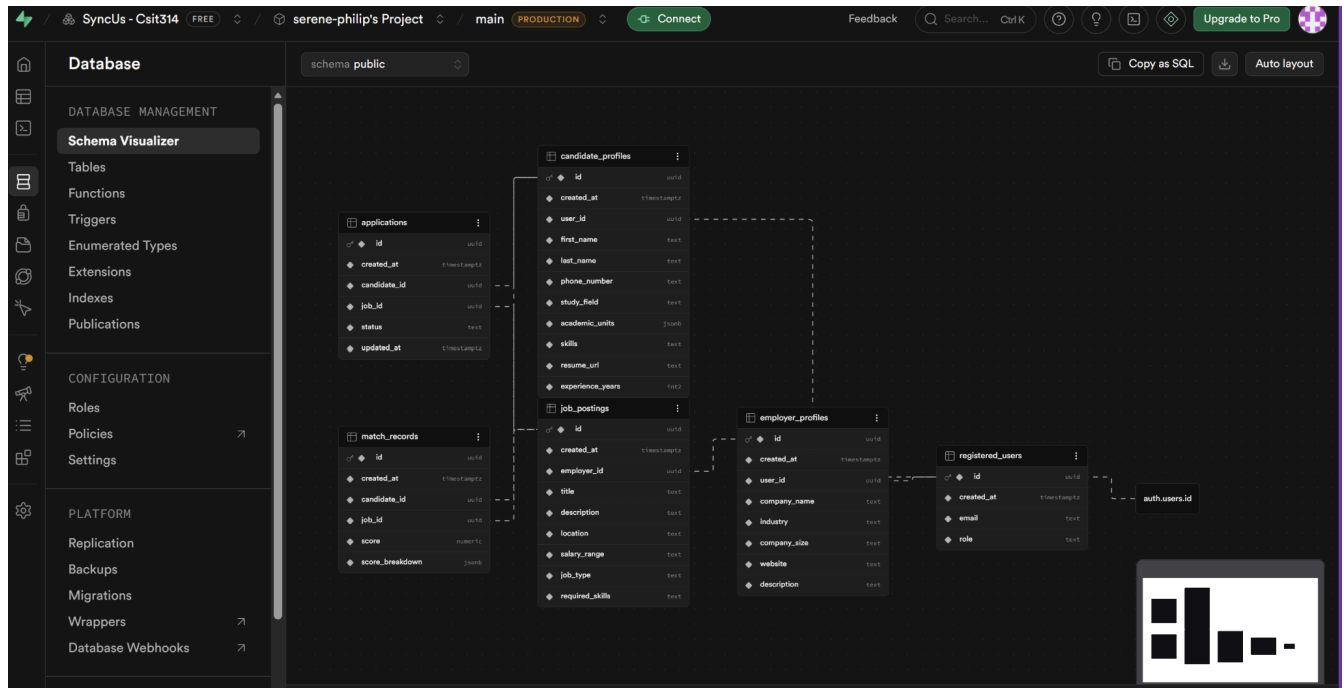


Figure 8: Initial Supabase Schema Visualiser

This structure ensures:

- **Separation of concerns** between frontend and backend.
- **Domain-driven development**, where each feature module aligns with system requirements.
- **Scalability**, allowing modules to be extracted into microservices in the future if required.

The development progress of each module is summarised in the table below, highlighting implementation status, key features, and API endpoints.

Module Implementation Summary

Module	Status	Key Features Implemented	Key API Endpoints
Accounts	Core complete; parsing in progress	Onboarding, profile management, resume upload, completeness scoring	POST /skill-sync/v1/accounts/onboarding - complete onboarding (role, basics) GET /skill-sync/v1/accounts/profile - get my profile PATCH /skill-sync/v1/accounts/profile - update profile POST /skill-sync/v1/accounts/resume - upload resume metadata / trigger parse GET /skill-sync/v1/accounts/resume - list my resumes GET /skill-sync/v1/accounts/completeness - profile completeness score
Jobs	CRUD complete; publishing implement	Job creation/editing, search/filter, publish/close workflow	POST /skill-sync/v1/jobs - create job (employer) PATCH /skill-sync/v1/jobs/{jobId} - edit job GET /skill-sync/v1/jobs/{jobId} - job detail GET /skill-sync/v1/jobs - search/filter jobs POST /skill-sync/v1/jobs/{jobId}/publish - publish draft

	ed		POST /skill-sync/v1/jobs/{jobId}/close - close posting
Matching	Core logic complete; optimisation in progress	Top-K recommendations, candidate ranking, match scoring	GET /skill-sync/v1/matching/recommendations - top-K matches for current user GET /skill-sync/v1/matching/jobs/{jobId}/candidates - ranked candidates for employer POST /skill-sync/v1/matching/recompute - force recompute for current profile/job GET /skill-sync/v1/matching/explanations/{matchId} - score breakdown (60/30/10)
Tracking	Core lifecycle implemented; pipeline view in progress	Application submission, tracking, state transitions	POST /skill-sync/v1/tracking/applications - apply to a job GET /skill-sync/v1/tracking/applications - list my applications GET /skill-sync/v1/tracking/applications/{applicationId} - application detail PATCH /skill-sync/v1/tracking/applications/{applicationId}/status - transition state GET /skill-sync/v1/tracking/jobs/{jobId}/pipeline - employer pipeline view POST /skill-sync/v1/tracking/applications/{applicationId}/notes - recruiter notes

6.4 Milestone-based Progress Tracking

A milestone-based approach is used to track project progress by comparing completed work against predefined milestones. This ensures the project remains aligned with the planned timeline and helps identify current and upcoming development priorities.

Milestone	Week	Description
M1: Requirements Completion	Week 4	All user stories and use cases finalized
M2: System Design Approval	Week 5	Architecture and UI confirmed
M3: Core Engine Implementation	Week 7	Matching algorithm functional
M4: Feature Completion	Week 10	All major features implemented
M5: System Integration	Week 11	Full system connected
M6: Testing Completion	Week 12	System validated with UAT
M7: Final Submission	Week 13	Project demo + report

Previous Milestone: M1 – Requirements Completion (Week 4)

This milestone has been successfully completed. User stories, use cases, and requirement analysis were finalized, establishing a clear project scope.

Current Milestone: M2 – System Design Approval (Week 5–6)

This phase is nearing completion. UI/UX designs, database schema, and system architecture (Modular Monolith) have been defined, with minor refinements remaining.

Upcoming Milestone: M3 – Core Engine Implementation (Week 7)

The next phase involves starting development, including backend setup, authentication, and initial implementation of the matching engine.

The project is currently on schedule, with documentation completed and design in its final stage. The team is prepared to begin development without delays.

7. Risk Analysis and Management

7.1 Risk Assessment Matrix

The matrix below rates all SyncUs project risks by Likelihood (rows) against Consequence severity (columns). Risk ratings are derived from the Queensland State Archives standard 5×5 matrix.

Risk ID	Risk Description	Likelihood	Impact (1–5)	Existing Risk Controls	Consequence Rating	Level of Risk	Priority Rating
T1	Cold Start Problem: The matching engine produces empty results at demo due to insufficient candidate/job data in the database.	Likely	5	Mock data seeding script (/db/seeds/) generates 30 candidate profiles and 20 job postings from Sprint 2 onward. Seed data version-controlled on GitHub.	Very High / Major	EXTREME	Critical
T2	Matching Algorithm Inaccuracy: 60/30/10 weighted scoring ranks unsuitable candidates above suitable ones due to poor skill tokenisation.	Possible	5	10 benchmark candidate-job pairs pre-validated manually. The algorithm must achieve ≥80% accuracy before being marked Done. HuggingFace JobBERT-v2 & all-MiniLM-L6-v2 on standby as NLP upgrade.	Very High / Major	HIGH	Critical
T3	RLS Misconfiguration: Incorrect Row-Level Security policies in Supabase expose candidate PII to unauthorised employers or block authorised feature access.	Unlikely	5	All RLS policies are peer-reviewed before frontend integration. Supabase Policy Simulator tested with Employer and Candidate roles. Mandatory item on integration testing checklist.	Extreme / Critical	HIGH	High
T4	Frontend-Backend Integration Failure: React frontend and Supabase backend developed in parallel cause API mismatches, schema conflicts, or JWT token errors.	Likely	4	Three-Tier Branching Strategy (Main → Dev → Feature). No merge to dev without a passed Pull Request and peer review. Sprint 8 (Week 11) is dedicated to full integration testing.	Very High / Major	HIGH	Critical

T5	Resume File Handling Failure: Malformed, oversized (>5MB), or disguised malicious files uploaded to Supabase Storage.	Possible	3	Dual-layer validation: client-side checks (type, size) and Supabase Edge Function re-validation before storage. Unit tests cover a 5.1MB file, .exe renamed as .pdf, and an empty file.	Medium / Moderate	MEDIUM	High
P1	Week 8 Requirement Change: Guaranteed scope change causes Strategic Drift; team continues building features that conflict with new requirements.	Almost Certain	4	Week 9 is designated as Buffer/Refactor Sprint. Could-Have features kept as descope targets. Impact Analysis Meeting convened within 24 hrs of announcement. Daily stand-ups during Weeks 8–9.	Very High / Major	EXTREME	Critical
P2	Team Availability Conflict: Unplanned absences during critical sprints (Sprints 3–4: matching engine) significantly reduce velocity.	Possible	4	Scrum Master monitors individual velocity via Taiga Burn-down Charts. 20% time buffer per task. Emergency stand-up within 24 hrs for task redistribution. 1-hr solo troubleshooting limit enforced.	Very High / Major	HIGH	High
P3	Scope Creep: Team invests sprint capacity in lower-priority features (resume templates, analytics) before the core matching engine is stable.	Likely	3	MoSCoW framework governs Product Backlog. Must-Have features locked into Sprints 2–7. Only the Product Owner may re-prioritise the backlog. Sprint velocity is reviewed weekly via the Burn-down Chart.	Medium / Moderate	HIGH	High
P4	Definition of Done Not Enforced: Untested or undocumented code enters the dev branch, accumulating Integration Debt that fails during Sprint 8–9 testing.	Possible	4	GitHub Pull Request template requires explicit DoD checklist sign-off (peer review, ≥80% unit test coverage, docs updated, AC verified) before any merge is permitted.	Very High / Major	HIGH	High
D1	Candidate PII Privacy Breach: Personally identifiable information (name, email, phone, resume) exposed to unauthorised employers; violates Australian Privacy Act 1988.	Unlikely	5	Contact info hidden behind RLS until the reciprocal match is confirmed in the Matches table. Resumes stored in a private Supabase bucket; accessed only via short-lived server-generated signed URLs.	Extreme / Critical	HIGH	High

D2	Data Loss via DB Migration: Incorrect DROP TABLE or ALTER COLUMN script destroys candidate profiles, job postings, or application records.	Unlikely	5	All schema changes are committed as versioned migration files in /db/migrations/ with PR review required. Changes validated on the dev Supabase project for one full sprint before applying to production.	Extreme / Critical	HIGH	High
E1	Supabase Service Outage: Backend-wide outage renders authentication, database, storage, and real-time features non-functional, including during demo.	Unlikely	5	General Feed uses a public PostgreSQL view for graceful degradation. Pre-demo SQL dump maintained as an emergency restore point. The Supabase status page is monitored before all demo sessions.	Extreme / Critical	HIGH	High
E2	GitHub Repository Corruption: Accidental force-push to main or account compromise destroys sprint code history.	Rare	5	Branch Protection Rules enabled: direct pushes and force-pushes to main blocked. All members maintain local clones. All merges require at least one approved PR.	Very High / Major	MEDIUM	Medium

7.2 Change Management: The Week 8 Pivot Protocol

The guaranteed requirement change at Week 8 is the most structurally significant risk to the SyncUs project. The team has designed an explicit **Week 8 Pivot Protocol** to ensure this change is absorbed as a managed Scrum event rather than a disruptive incident.

Phase 1: Pre-Change Preparation (Sprints 2–7)

The team deliberately front-loads the highest-risk, highest-complexity components, specifically the 60/30/10 matching engine and the Supabase RLS architecture, into Sprints 3–4. By Week 7, the core system infrastructure is stable and tested. This ensures that whatever the Week 8 change introduces, it modifies surface-level features (UI, new fields, additional filters) rather than requiring a restructuring of the database schema or matching logic.

The Product Backlog is also maintained with a dedicated "Change Buffer" section, a list of Could-Have features (e.g., resume template generation, hiring team collaboration) that are intentionally unscheduled. These serve as displacement targets: if the Week 8 change introduces a new Must-Have requirement, a Could-Have item is immediately descope to protect the timeline.

Phase 2: Impact Analysis Meeting (Week 8, Day 1)

Within 24 hours of the announcement, the team decomposes the new requirement into user stories, maps dependencies, re-estimates effort via Planning Poker, and the Product Owner re-prioritises the backlog accordingly.

Phase 3: Intensified Collaboration Window (Weeks 8–9)

During this two-week window, the team transitions from weekly stand-ups to **Daily Stand-ups** (Monday–Friday, 15 minutes maximum) to prevent obsolete development.

Week 9 is a dedicated **Buffer/Refactor Sprint** with no new features, only for integrating the change and resolving regressions.

Phase 4: Verification and Resume (Weeks 10–13)

By Week 10, the system returns to its standard sprint cadence. The Buffer Sprint ensures that Sprints 7–10 (Job & Profile Features, Search & Filtering, Integration, Testing) are not compressed. All sprint goals from Week 10 onward are re-estimated using the updated velocity data captured in the Taiga Burn-down Chart from the pivot window.

The following table summarises the protocol's decision gates:

Gate	Trigger	Decision	Owner
G1	Week 8 change announced	Call Impact Analysis Meeting within 24 hrs	Scrum Master
G2	New stories estimated	Descope Could-Have items if total story points exceed sprint capacity	Product Owner
G3	End of Week 9 Buffer Sprint	Verify the <code>dev</code> branch passes all integration tests before resuming feature work	Dev Team
G4	Week 12 UAT	Confirm the new requirement satisfies the Acceptance Criteria before final merge to <code>main</code>	Product Owner

This structured protocol ensures the Week 8 change is treated as a planned backlog refinement event within the Scrum framework, preserving delivery confidence without sacrificing code quality or system stability.

References

Drumond, C 2024, What is Scrum? Breaking down the Agile framework, Atlassian, viewed April 17, 2026 from <<https://www.atlassian.com/agile/scrum>>

Kulkarni, V 2023, How to effectively deal with and leverage ambiguity as product managers, Mind the Product, viewed 14 April 2026, <<https://www.mindtheproduct.com/ambiguity-as-product-managers/>>.

Wondrasek, JA 2026, What Is a Modular Monolith and How Does It Combine the Best of Both Architectural Worlds - SoftwareSeni, SoftwareSeni. viewed April 14, 2026 from <<https://www.softwareseni.com/what-is-a-modular-monolith-and-how-does-it-combine-the-best-of-both-architectural-worlds/#:~:text=Is%20a%20modular%20monolith%20just,guidance%20for%20choosing%20between%20patterns.>>